

CSE 256 Midterm Exam - Winter 2026 (70 Points)

March 3, 2026 (*Solutions*)

Instructions

- You are **allowed** one double-sided A4 note sheet. You are **not allowed** calculators or other electronic devices.
- **Partial credit** will be given for explanation questions. It is to your advantage to show work.
- **The penalty for academic dishonesty will be a course grade of F** and a referral of the case to UCSD Academic Integrity Office. Further penalties, including suspension or expulsion from the University may be imposed by that office. By turning in this exam with your name on it, you are certifying that this is yours and yours alone. You must not copy from others, and you must not allow others to copy from you.

Name: _____

Student ID: _____

If a question seems vague or under-specified to you, make an assumption, write it down, and solve the problem given your assumption.

Question	Points	Score
Part 1: Multiple Choice	40	
Part 2: Transformers	20	
Part 3: Post-Training	10	
Total	70	

Note on Solutions: the provided solutions are not exhaustive, full credit might be given for other valid explanations.

Part 1: Multiple Choice Questions (40 points) - Recommended 25 Minutes

Multiple-Choice Instructions Fill in the circle(s) next to the letter(s) of your choice (e.g., $\bullet$). No explanations are required. Each question is worth 5 points and has **exactly one or two** correct answers. Full credit (5 points) requires selecting all correct options and no incorrect options. Each incorrect selection or missed correct selection results in a 2.5-point deduction (up to 5 points total).

1. (5 points) Consider a Transformer residual connection with input $\mathbf{x}$ and output $\mathbf{y}$, where the computation is given by:

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x}).$$

Which statements are true?

- ☒ During backpropagation, the gradient includes an identity term that allows signal to flow even if $\frac{\partial F}{\partial \mathbf{x}}$ is small.
- ☐ Residual connections are primarily used to facilitate fast inference.
- ☒ If $F(\mathbf{x}) \approx 0$, the block behaves approximately like the identity function.
- ☐ Without residual connections, stacking many Transformer layers make the model unable to represent the identity function even if the training task requires it based on the training data.

2. (5 points) The perplexity of a language model on a dataset of M tokens is defined as

$$\text{PP} = 2^{-\left(\frac{1}{M} \sum_{i=1}^M \log_2 p(x_i)\right)}.$$

where $p(x_i)$ is the probability assigned by the model to the i -th token and M is the total number of tokens. Which statements are true?

- ☐ Perplexity is 0 if the model assigns probability 1 to every token in the dataset.
- ☒ Lower perplexity indicates a better language model on the evaluated dataset.
- ☒ If a model assigns uniform probability $p(x_i) = \frac{1}{|V|}$ over a vocabulary of size $|V|$, then its perplexity is $|V|$.
- ☐ Perplexity can take negative values if the log-probabilities are negative.

3. (5 points) For a softmax classifier with cross-entropy loss, the gradient with respect to the weights of class $j \neq y_i$ is

$$\frac{\partial L_i}{\partial W_j} = p(j | x_i) x_i.$$

If $p(y_i | x_i) \approx 0$ and one incorrect class k has $p(k | x_i) \approx 1$, what is the approximate SGD update to W_k ?

- ☒ $W_k \leftarrow W_k - \eta x_i$
- ☐ $W_k \leftarrow W_k + \eta x_i$
- ☐ W_k remains unchanged
- ☐ W_k moves toward W_{y_i}

4. (5 points) Which models are susceptible to the vanishing gradient problem?
- ☐ A 1-layer feed-forward neural network.
 - ✓ A very deep feed-forward neural network.
 - ✓ A recurrent neural network (RNN) applied to a long sequence.
 - ☐ Word2vec Skip-Gram.
5. (5 points) Large language models can perform new tasks using zero-shot, few-shot, or chain-of-thought (CoT) prompting, without updating their parameters. Which statements help explain why this is possible?
- ✓ The model was pretrained on large and diverse text corpora, allowing it to internalize many task patterns during training.
 - ☐ The model updates its parameters while processing the prompt.
 - ✓ The prompt acts as conditioning context, allowing the model to adapt its behavior via next-token prediction.
 - ☐ Chain-of-thought prompting modifies the model architecture to enable reasoning.

6. (5 points) Consider Layer Normalization applied to a token representation

$$\mathbf{h} \in \mathbb{R}^d, \quad \text{LN}(\mathbf{h}) = \gamma \odot \frac{\mathbf{h} - \mu}{\sigma} + \beta.$$

Which statements are true? (Oversight here in that there are three correct options, full credit is given for selecting any two of the three or even all three)

- ✓ LayerNorm is used to stabilize training by ensuring that unhelpful fluctuations in the scale of $\mathbf{h}$ do not cause large changes in the output.
 - ✓ LayerNorm is applied both during training and inference time.
 - ☐ LayerNorm normalizes across tokens in the sequence dimension.
 - ✓ Without the learned parameters γ and β , the layer cannot represent the identity mapping where normalization can be undone and the output equals the input.
7. (5 points) Suppose softmax is used as the activation function of a hidden layer with neurons $h_1, h_2, \dots, h_n$ where $n = 100$, thus softmax is normalizing across all 100 hidden units. Which statements are true?
- ☐ The softmax function applied to the hidden layer is equivalent to applying a sigmoid non-linearity to each unit independently.
 - ✓ The hidden representation values in a layer will all be non-negative and sum to 1.
 - ✓ Hidden units would compete with each other, since increasing one activation decreases others.
 - ☐ Each hidden unit would activate independently, similar to ReLU.
8. (5 points) Modern large language models often use subword tokenization methods such as Byte Pair Encoding (BPE). Which statements are true?
- ☐ Using a special token for unknown words allows the model to handle out-of-vocabulary words just as well as BPE.
 - ✓ BPE allows rare words to be represented as combinations of more frequent subword units.
 - ✓ A larger vocabulary increases the computational cost of the softmax layer during training and inference.
 - ☐ BPE guarantees that every word in the language appears as a single token.

Part 2: Transformers (20 points) - Recommended 30 Minutes

2A) (6 points) Let $X \in \mathbb{R}^{L \times d_{\text{model}}}$ be an input sequence of length L . In multi-head self-attention with h heads and $d_k = d_{\text{model}}/h$, each head uses

$$W_q^{(i)}, W_k^{(i)}, W_v^{(i)}, \quad W_o$$

Define

$$Q_i = XW_q^{(i)}, \quad K_i = XW_k^{(i)}, \quad V_i = XW_v^{(i)},$$

$$A_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right), \quad \text{head}_i = A_i V_i,$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_o.$$

If the sequence length L changes, which tensor shapes change and which remain unchanged? Justify your answer by giving the shapes of Q_i, K_i, V_i, A_i , and head_i , those of the weight matrices, and explaining how they depend on L or not.

The shapes of the weight matrices are as follows: - $W_q^{(i)}: (d_{\text{model}}, d_k)$ - $W_k^{(i)}: (d_{\text{model}}, d_k)$ - $W_v^{(i)}: (d_{\text{model}}, d_k)$ - $W_o: (h \cdot d_k, d_{\text{model}})$

And then, the computed tensors have the following shapes:

- $Q_i: (L, d_k)$
- $K_i: (L, d_k)$
- $V_i: (L, d_k)$
- $A_i: (L, L)$
- $\text{head}_i: (L, d_k)$

Thus the shapes of Q_i, K_i, V_i, A_i , and head_i depend on the sequence length L , while the shapes of the weight matrices do not depend on L and remain unchanged when L changes as those are fixed parameters of the model once it is trained.

2B) (6 points)

State of the art LLMs such as Llama 3 use *Grouped Query Attention (GQA)*, a variant of standard multi-head attention (MHA). In standard MHA with h heads:

$$Q_i = XW_q^{(i)}, \quad K_i = XW_k^{(i)}, \quad V_i = XW_v^{(i)}, \quad i = 1, \dots, h.$$

Each head has its own query, key, and value projections.

In GQA:

- There are h query heads,
- but only $g < h$ key/value heads,
- each KV head is shared across $\frac{h}{g}$ query heads.

i) (3 points) Compared to standard MHA, GQA shares keys and values across multiple query heads. What type of representational flexibility might be reduced in GQA compared to standard MHA due to this sharing? Briefly explain your answer. Llama 3 is still powerful and achieves strong performance despite this reduced flexibility. Why might GQA not significantly hurt model performance in practice despite the reduced flexibility?

In MHA, each head can learn to attend to a unique set of features in the keys/values. In GQA, those query heads are forced to project their questions onto the same answer/feature space (the shared K/V), this can potentially limit the diversity of attention patterns that can be learned.

However, in practice this reduction might not hurt performance because:

- Large models are often overparameterized, so sharing KV heads might remove redundant capacity rather than essential model capacity.
- Multiple query heads can still discover different relationships from the same shared KV heads by applying different learned weights, that is, viewing the same subspace from different angles.

ii) (3 points) Consider the KV cache memory cost during autoregressive generation. For sequence length T , number of query heads h , number of KV heads g , and head dimension d_k :

$$\text{MHA KV memory} = O(T \cdot h \cdot d_k)$$

$$\text{GQA KV memory} = O(T \cdot g \cdot d_k)$$

$$\text{Reduction factor} = \frac{g}{h}$$

Does this reduction primarily benefit training, inference, or both? Explain your answer. Why does this reduction become especially important for large models?

This reduction primarily benefits inference. The KV cache is used during autoregressive generation to store past keys and values, which only occurs at inference time. While training also computes attention, it does not rely on a KV cache due to the use of teacher forcing.

This reduction becomes important for large models since the head dimension d_k is often large, and the number of heads h can also be large in big models, as well as the number of layers which each have their own attention heads.

2C) (2 points) Suppose we choose to do autoregressive generation with a BERT model pretrained with masked language modeling (MLM) instead of a decoder-only model pretrained with next-token language modeling (NLM).

Why can we not use KV caching in BERT for autoregressive generation at inference time but can use it in a decoder-only model? Explain your answer.

BERT uses bidirectional attention, the representation of every token depends on every other token in the sequence (past and future). If you generate a new token $n + 1$ and append it to the sequence, the attention weights and hidden states for all previous tokens (1 to n) must be re-calculated because they can now attend to the new token. This violates the requirement for caching: that the past must remain static.

Even though the keys and values are projections of the inputs, in higher layers, these inputs are hidden states from prior layers, and in BERT, those hidden states (outputs of layer l and inputs to layer $l + 1$) change.

2D) (6 points) You are given a pretrained BERT encoder. The masked language modeling (MLM) head used during pretraining has been removed. You would like to fine-tune the model for downstream supervised tasks by adding task-specific output layers.

Describe precisely how you would fine-tune BERT for each of the following tasks.

In your answer, specify:

- How the input is formatted and presented to BERT, describe any special tokens used.
- Which hidden representation(s) from BERT are used as input to the task-specific head.
- The architecture of the task-specific head.

i) (3 points) A standard single-sentence classification task (e.g., sentiment analysis with C classes).

- **Input formatting:** We would format the input sentence by adding a special [CLS] token at the beginning of the sentence. This allows BERT to learn a representation for the entire sentence in the [CLS] token.
- **Hidden representation:** We would use the hidden representation corresponding to the [CLS] token from the final layer of BERT as input to the task-specific head.
- **Task-specific head:** The task-specific head would be a simple softmax classifier, which consists of a linear layer that maps the [CLS] representation to a C -dimensional output, followed by a softmax activation to produce class probabilities.

ii) (3 points) A multiple-choice question answering task. Each example consists of a question and $K = 4$ candidate answers, exactly one of which is correct.

- **Input formatting:** For each candidate answer a_k ($k = 1, \dots, K$), we construct a sequence

[CLS] Question [SEP] Answer _{k} [SEP]

BERT is run once for each question–answer pair, producing K separate forward passes when $K = 4$.

- **Hidden representation:** From each forward pass, we take the final hidden representation of the [CLS] token as a summary vector for that question–answer pair.
- **Task-specific head:** A linear layer maps each [CLS] vector to a scalar score. The K logits are passed through a softmax to obtain a probability distribution over the answer choices. The model is trained using cross-entropy loss over the K options.

Alternative (not used in practice) approach:

- **Input formatting:** We would format the input by concatenating the question and each candidate answer into a single sequence, separated by special tokens. For example, we could use the format: [CLS] Question [SEP] Answer_1 [SEP] Answer_2 [SEP] Answer_3 [SEP] Answer_4 [SEP]. This way, BERT can learn to attend to the question and each answer option together.
- **Hidden representation:** We would use the hidden representation corresponding to the [CLS] token from the final layer of BERT as input to the task-specific head, as it can capture information about the entire question and answer options.
- **Task-specific head:** The task-specific head would be a linear layer that maps the [CLS] representation to a 4-dimensional output, followed by a softmax activation to produce probabilities for each of the 4 answer options. The model would be trained to maximize the probability of the correct answer.

This approach is not used because it forces a single CLS token to learn a ranking function, which is harder than learning a compatibility function for each question-answer pair separately. It also allows for cross-answer interference, where the model might learn to compare answer options against each other rather than just against the question, which can be bad for generalization.

Part 3: Post-Training of Language Models (10 points) - Recommended 15 Minutes

After pretraining, large language models undergo a post-training stage to improve helpfulness and alignment.

Supervised fine-tuning (SFT) maximizes token-level likelihood:

$$\max_{\theta} \sum_t \log P_{\theta}(y_t \mid x, y_{<t})$$

In contrast, RLHF optimizes expected reward under a learned preference model:

$$\max_{\pi_{\theta}} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} [R_{\phi}(x, y)].$$

3A) (2 points) SFT assumes the existence of a single “correct” response. State one type of task or application where this assumption might be problematic? Briefly explain your answer.

Answer: One type of task where the single “correct” response assumption of SFT is not desirable is open-ended generation (e.g., story generation, poetry). In such tasks, there can be many valid and diverse responses that are all equally good, and enforcing a single correct answer can limit the model’s creativity and lead to mode collapse, where the model generates the same or similar responses no matter the open-ended nature of the task.

3B) (3 points) Why can RLHF optimize sequence-level properties that SFT cannot? Give an example of a sequence-level property that RLHF can optimize but SFT cannot.

RLHF can optimize sequence-level properties because it assigns a reward to the entire generated response, allowing the model to learn from holistic human judgments. In contrast, SFT trains on token-level likelihoods of reference outputs and cannot directly optimize properties of the full sequence.

For example, RLHF can optimize helpfulness or harmlessness of a response, such as rewarding a model for refusing unsafe requests, which depends on evaluating the overall response rather than matching specific target tokens.

3C) (3 points) Suppose we skip SFT and directly apply RLHF starting from a base language model. How might this affect the RLHF pipeline for InstructGPT that we saw in class, which uses SFT as a first step, following by reward model training and then RLHF? Briefly explain your answer.

Answer: Skipping SFT and applying RLHF directly to a base model is inefficient because base models are sequence completers, not assistants. Without a supervised warm-up, can't follow instructions, making it difficult for a reward model or human to provide meaningful feedback on its initial outputs.

SIDE NOTE:

DeepSeek's experiments with **R1-Zero** (pure RL) proved that while reasoning can emerge without SFT, the results suffer from:

- **Poor Formatting:** Responses are disorganized and difficult to read.
- **Unstable Learning:** Without an initial template, the RL process lacks a stable baseline for optimization.

DeepSeek concluded that an SFT phase, using even a small amount of high-quality data, is needed for effective RLHF.

3D) (2 points) Despite supervised fine-tuning and RLHF post-training, large language models still frequently hallucinate (e.g., fabricating citations in academic writing).

List and explain two distinct factors related to: training objectives; optimization; and/or inference procedures such as decoding/sampling methods, that contribute to hallucination in large language models.

- **Probabilistic Decoding & Nucleus Sampling:** Inference methods like *top-p* or *top-k* sampling introduce randomness to increase creativity and avoid repetition. This randomness can cause the model to sample a low-probability but linguistically plausible token.
- **Supervised Fine-Tuning (SFT) Objective:** SFT can lead to confident but incorrect responses (hallucinations) because for each instruction, it forces the model to predict the corresponding ground truth answer, regardless of whether the model actually has the knowledge or reasoning ability to generate a correct response. This can lead to the model learning to predict things for which it has no basis, resulting in confident but incorrect outputs.
- **Maximum Likelihood Estimation (MLE) & Exposure Bias:** During training, models are optimized to predict the next token based on ground-truth prefixes. However, during inference, they predict based on their own previously generated (and potentially erroneous) tokens. This exposure bias, allows a single small error to compound, and the model does not learn to recover from its own mistakes, potentially leading to hallucination.